

ソーラーカー MPC-EMS 完全導入・運用手順書

Windows 11 / WSL2 / Ubuntu 22.04 / ROS 2 Humble

2026-07-16

目次

1 対象、配布物、安全境界	1
2 Windows 11 への初回導入	2
2.1 ZIP 取得	2
2.2 WSL2 導入	2
2.3 導入成功判定	2
3 Ubuntu 22.04 への初回導入	2
4 package 構造と一つの正本	2
5 全 action と 4 mode	3
6 空 package から車両モデルを完成させる	3
6.1 入れる根拠資料	3
6.2 入れる時系列	4
6.3 MLE 実行	4
7 route、schedule、weather の投入	4
8 大会全体 offline simulation	4
9 実測 mode	5
10 WiFi、Raspberry Pi、マイコン設定	5
11 live WiFi の起動と 1 周期	6
12 race-day release gate	6
13 異常時の復旧	6
14 出力の所在と追跡	6
15 教材と最終確認	7

本書は、GitHub から初めて ZIP を取得する時点から、車両同定、天候取得、大会全体シミュレーション、実測、live WiFi、結果保存、障害復旧までを、初見のチーム員だけで再現するための手順書である。

1 対象、配布物、安全境界

動作保証対象は Ubuntu 22.04 + ROS 2 Humble である。Windows 11 では WSL2 上の Ubuntu 22.04 を使う。Native Windows ROS、Ubuntu 24.04/Jazzy は、この release の検証対象ではない。

配布物は次の二つである。

- MPCEMS_YATA_2027: YATA の同定例、採用 map、報告書を含む全機能版。
- MPCEMS_base: 同じ実行機能を持つが、車両・大会データが空の雛形版。

本システムが送るものは速度助言値であり、直接のトルク指令ではない。運転者、車両側速度上限、電装保護、非常停止が常に優先する。未同定の空 package を live 送信へ使ってはならない。

公式参照先は WSL 導入、ROS 2 Humble Ubuntu 導入、GitHub ZIP 取得 である。

2 Windows 11 への初回導入

2.1 ZIP 取得

GitHub の **Code > Download ZIP** を選び、C:\solar\MPCEMS_YATA_2027 のような 短いローカルパスへ展開する。OneDrive 上でも動くが、大量の CSV、build、PDF 生成では 同期負荷が増えるため、運用 PC では短いローカルパスを推奨する。

2.2 WSL2 導入

管理者 PowerShell で一度だけ実行する。

```
wsl --install -d Ubuntu-22.04
```

再起動を要求されたら再起動し、Ubuntu を一度起動して Linux ユーザーを作る。展開先で通常の PowerShell を開き、次を実行する。

```
Set-ExecutionPolicy -Scope Process Bypass
.\Install-SolarSim.ps1
.\SolarSim.ps1 -Action audit
```

Install-SolarSim.ps1 は Windows のパスを WSL パスへ変換し、Ubuntu 側の scripts/bootstrap_ubuntu_humble.sh へ渡す。途中で sudo password を求められる。

2.3 導入成功判定

次がすべて成功しなければ先へ進まない。

```
wsl -d Ubuntu-22.04 bash -lc "source /opt/ros/humble/setup.bash; ros2 --help >/dev/null"
.\SolarSim.ps1 -Action build
.\SolarSim.ps1 -Action audit
```

build が install/setup.bash を作り、audit が inventory、静的監査、pytest を通す。失敗時は最後の 80 行を保存し、依存を無視して起動しない。

3 Ubuntu 22.04 への初回導入

ZIP を展開するか clone し、repository root で次を実行する。

```
cd ~/solar/MPCEMS_YATA_2027
bash scripts/bootstrap_ubuntu_humble.sh
bash scripts/solar_control.sh audit sim config/solar/bwsc_2027_demo.yaml
```

bootstrap は ROS 公式 repository、Humble Desktop、科学計算 Python、Graphviz、rqt-graph、chrony を導入し、rosdep、colcon build、pytest まで行う。CasADi だけは Jammy の rosdep key 差を避けて pip user 領域へ入れる。再実行しても既存 ROS 設定を壊さない。

通常の shell から直接 ROS command を使う場合は、その terminal ごとに次を読む。

```
source /opt/ros/humble/setup.bash
source install/setup.bash
```

4 package 構造と一つの正本

全 action は一つの profile.yaml を受け取る。相対 path は profile の所在 directory を 基準に解決され、実行時には解決済み profile が結果と一緒に保存される。

profile block	内容
paths	route、weather、schedule、全採用 map
runtime	timezone、dashboard host/port
logging	live/measurement 保存先と rate

profile block	内容
simulation	開始 UTC、初期状態、version 出力先
measurement	distance/grade 実測条件
identification	raw/evidence/fit 出力先
live	weather API、autocal、command/WiFi bridge
model	質量、CdA、Crr、補機、電池、電流・温度上限
mpc	上位/下位 horizon、終端帯、目的関数、solver

主要 source の責務は次の通りである。

source	責務
model.py	力、効率 map、PV、pack、IV、SoC、熱状態
upper_cost.py / upper_solver.py	上位目的・制約と global/local 探索
mpc_node.py	remaining-course 上位 MPC と 1 Hz 下位 MPC
solar_sim.py	offline 全コース最適化、replay、全 CSV/manifest
telemetry_text_bridge_node.py	UDP 解釈、時刻 gate、filter、ROS topic 化
speed_command_bridge_node.py	timeout、平滑化、変化率、量子化、安全送信
solar_logger_node.py	同期済み live/measurement CSV
dashboard_node.py	一画面 dashboard と API

生成済み結果を上書き編集しない。profile を説明的な別名で保存し、latest_simulation_run.json と解決済み profile で結果を特定する。

5 全 action と 4 mode

Windows 形式は次である。

```
.\SolarSim.ps1 -Action <action> -Mode <mode> -Profile <profile.yaml>
```

Ubuntu 形式は次である。

```
bash scripts/solar_control.sh <action> <mode> <profile.yaml>
```

action は up/build/start/stop/restart/status/graph/simulate/forecast/identify/fit/ learn/audit/package/blank/log、mode は sim/measure/live/live_wifi である。

mode	launch	実際に起動する機能
sim	solarcar_sim.launch.py	GPS 模擬、状態伝播、階層 MPC、dashboard
measure	solar_measurement.launch.py	preflight、distance、grade、logger、dashboard
live	solar_race_live.launch.py	weather、autocal、MPC、安全 command、logger
live_wifi	solar_race_live_wifi.launch.py	live 全部、UDP telemetry、風補正

up は build、古い同 package node 停止、選択 launch 起動を順に行う。PowerShell 入口は dashboard 応答を待ち、実 node が生きている間に rqt graph を採取する。

```
.\SolarSim.ps1 -Mode live_wifi -Action status
.\SolarSim.ps1 -Mode live_wifi -Action log
.\SolarSim.ps1 -Mode live_wifi -Action graph
.\SolarSim.ps1 -Mode live_wifi -Action stop
```

停止中に graph exporter だけが表示されるのは正常である。必ず up 後に graph を取る。

6 空 package から車両モデルを完成させる

6.1 入れる根拠資料

project_packages/<vehicle>/data/identification/evidence へ、motor/controller の 公式試験、eco/power/regen 効率、panel/MPPT 仕様、cell 放電・OCV 曲線、Rint 対 SoC/温度、補機実測、車体諸元を置く。根拠のない滑らかな

map を最初から fit してはならない。まず商品・試験・物理式から grounded base map を作り、その後に 小さい scale/shape 補正だけを許す。

6.2 入れる時系列

data/identification/raw へ最低限、UTC 時刻、速度、距離または GNSS、pack 電圧・電流・温度、PV 電力を置く。battery pulse/rest、panel sweep、coast-down、定速、勾配、独立 SoC/容量 anchor、イベント記録を別 session として残す。

電流符号は放電正、充電負に統一する。停止、control stop、事故、センサ異常を 同じ flag にせず、event YAML で理由と時刻・地点を明示する。

6.3 MLE 実行

```
.\SolarSim.ps1 -Action fit -Profile project_packages\<vehicle>\profile.yaml
```

順序は、入力 QA、grounded map、segment 分割、PV fit、battery OCV/Rint/capacity fit、motion fit、11 次元 joint refinement、bounded map shape fit、historical replay、day 別 hold-out、終端 anchor、採否判定である。

確認対象は *_generic_fit_summary.yaml、replay_validation.csv、adopted maps、PDF 報告書である。training RMSE だけが低くても、時刻ずれ、同一日の過適合、2831 km 終端、独立日、物理 bound が不合格なら採用しない。採用時だけ canonical profile の model/map path が更新され、sim と live が同じ車両を読む。

YATA では 25 直列、race-use 上限 4.35 V/cell なので、model.V_max は $25 * 4.35 = 108.75$ V である。OCV map 最大値は 108.75 V 以下、YATA profile の V_max は 108.75 V に一致しなければならない。audit は上下どちらの不整合もエラーにする。

手計算は docs/mle_hand_calculation_workbook の問題編、解答用紙、解答編を使う。

7 route、schedule、weather の投入

route waypoints/profile、区間速度上限、公式 drive window、control stop、timezone、全コース距離、開始 UTC、soc0 を profile へ入れる。historical replay では同時刻・同地点の archive 観測を使い、将来 sim では選択 forecast を使う。

```
.\SolarSim.ps1 -Action forecast -Profile project_packages\<vehicle>\profile.yaml
```

forecast CSV は sim の全 UTC 範囲を覆う必要がある。live では chase GNSS 地点で取得した raw forecast を保存してから、観測風補正を別 CSV へ書く。nominal plan は最尤天候を そのまま用い、日数とともに増える不確かさは upper/lower risk plan へ分離する。

8 大会全体 offline simulation

```
.\SolarSim.ps1 -Action simulate -Profile project_packages\<vehicle>\profile.yaml
```

名目目的は制約付き最短時間である。

$$\min_u T(u), \quad z_{\min} \leq z_k \leq z_{\max}, \quad z_{\min} \leq z_N \leq z_{\min} + \varepsilon.$$

さらに速度、drive window、停止、電圧、放電/充電電流、温度を守る。「使用可能 energy を残さない」は物理 SoC 0 ではなく、終端 SoC を安全下限から soc_finish_tol 以内へ入れる意味である。

detail CSV は各 step について、時刻・地点、route/weather、全抵抗、map 補間効率、PV raw/MPPT、wheel/DC/regen/aux/pack power、OCV/Rint/Rline、判別式、I/V、SoC/温度、step/cumulative energy、全 scalar parameter、全 map path を持つ。exact/fine profile では全列を保ったまま gzip 圧縮し、拡張子は *.csv.gz となる。pandas.read_csv(path) で直接読めるため、展開用の数 GB 空き容量は不要である。

manifest で次を確認する。

- finish_reached=true で、公式全コース距離へ到達した。
- terminal_energy_band_met=true である。
- 電圧、電流、温度、SoC、schedule 違反がない。
- power_balance_residual_w が丸め誤差程度である。
- solver status と証明範囲が明記される。

有限 grid 全列挙は、宣言した離散速度集合内では数学的な大域証明になる。SHGO と 局所改善は連続 bound を探索するが、有限回の結果だけを無条件の連続大域証明とは 呼ばない。両者の status を manifest で分離する。

9 実測 mode

```
.\SolarSim.ps1 -Mode measure -Action up -Profile project_packages\<vehicle>\profile.yaml
.\SolarSim.ps1 -Mode measure -Action status
.\SolarSim.ps1 -Mode measure -Action graph
.\SolarSim.ps1 -Mode measure -Action stop
```

動き出す前に、UTC 同期、GNSS fix、停止速度、電流符号、pack 電圧、logger の新 CSV を確認する。定速、coast-down、勾配、pulse/rest、panel sweep を別 run にし、run 中の 人為操作、停止理由、機器切替を event manifest へ記録する。

終了後は CSV、event、resolved profile、rqt graph、時刻同期結果を同じ run directory へ 保存する。Excel で原本を直接編集せず、修正版を別名にする。

10 WiFi、Raspberry Pi、マイコン設定

推奨固定 IP は制御 PC 192.168.50.10、solar Pi 192.168.50.21、chase Pi 192.168.50.22 である。PC は UDP 52001 で受信し、solar/chase Pi は command を 52002/52003 で受信する。private race LAN のこの UDP だけを firewall 許可する。

全機器で chrony/NTP を有効化し、開始前に UTC 差が 1 秒未満であることを確認する。vehicle Pi は UTF-8 JSON を 1-10 Hz で PC へ送る。

```
{
  "type": "vehicle_state",
  "timestamp_utc": "2027-08-28T01:23:45.120Z",
  "speed_kmh": 72.3,
  "soc": 0.83,
  "batt_temp_c": 34.2,
  "batt_current_a": 8.5,
  "batt_voltage_v": 97.6,
  "solar_power_w": 410.0,
  "lat": -22.0,
  "lon": 133.0,
  "alt_m": 612.4,
  "course_deg": 176.0
}
```

chase Pi は次を送る。

```
{
  "type": "chase_state",
  "timestamp_utc": "2027-08-28T01:23:45.120Z",
  "lat": -22.0,
  "lon": 133.0,
  "alt_m": 610.2,
  "wind_speed_ms": 6.5,
  "wind_dir_deg": 121.0,
  "course_deg": 176.0
}
```

単位は SoC 0-1、km/h、W、A、deg、WGS84 で、放電電流が正である。PC は時刻なし、5 秒超の古い packet、2 秒超の未来、重複、順序逆転を filter 前に拒否する。

PC から両 Pi へ返る形式は次である。

```
{
  "type": "planner_command",
  "timestamp_utc": "2027-08-28T01:23:46Z",
  "planner": {
    "speed_cmd_kmh": 71.2,
    "upper_speed_cmd_kmh": 72.0,
    "drive_mode": "eco"
  },
  "vehicle": {
    "speed_kmh": 70.9,
    "soc": 0.829,
```

```

    "s_km": 1200.02
}
}

```

solar_power_w にはマイコン側で係数を掛けない。送る値は DC bus の未補正実測値で、PC 側 bridge が profile で同定済みの solar_measurement_gain を一度だけ適用する。送信側と受信側の両方で補正すると、PV を過小または過大評価して SoC 計画を壊す。

マイコンは UDP 欠落時に過去 command を保持して加速してはならない。独立 timeout、速度上限、電流/電圧保護、driver override を持たせ、UDP を直接 torque へ変換しない。実装例は scripts/wifi_vehicle_sender_example.py、wifi_chase_sender_example.py、templates/network にある。

11 live WiFi の起動と 1 周期

release gate 通過後に起動する。

```
.\SolarSim.ps1 -Mode live_wifi -Action up -Profile project_packages\<vehicle>\profile.yaml
```

1 周期では、UDP 受信、UTF-8/JSON 解釈、UTC gate、range/rate filter、ROS publish、distance/grade/wind 更新、状態予測、下位 1 Hz MPC、安全 command bridge、UDP 送信、logger/dashboard 更新を同じ時系列で行う。上位 remaining-course MPC は設定周期または forecast 更新 trigger で再計画する。

command bridge は startup hold、input timeout、low-pass、加減速上限、deadband、0.1 km/h 量子化、drive mode 最低保持時間を順に適用する。weather/autocal は versioned raw を保存し、grounded model を bound なしで上書きしない。

dashboard で最低限、実速度、上下位 command、距離、SoC、V/I、PV、wind、通信 age、planner status、警告が一画面にあることを確認する。logger の時刻が増えない、bridge status が rejected、planner が infeasible の場合は出発しない。

12 race-day release gate

区分	合格条件
source	audit/pytest 成功、release commit/hash 固定
model	evidence、hold-out、終端 anchor、物理 bound 合格
race	全距離、schedule、timezone、stop、speed limit 確認
simulation	finish、終端 energy 帯、全制約、detail balance 合格
ROS	4 mode 起動、実 node rqt graph、重複 node なし
network	2 Pi、NTP、fresh/stale/future/duplicate UDP 試験
operation	dashboard/logger、disk、電源、shadow road test 合格

開始直前に profile hash、soc0、route/weather coverage、map provenance、remote IP/port を二人で読み合わせる。出発後に canonical profile を編集しない。

13 異常時の復旧

- telemetry 欠落: bridge の safe speed へ遷移させ、time/network を直す。gate を無効にしない。
- forecast 失敗: 最後の valid version を保持し、age を表示する。空 CSV へ差し替えない。
- planner infeasible: 安全速度または停止へ移り、制約をその場で緩めない。
- node 終了: log 保存、stop、原因修正、up。launch を二重起動しない。
- disk 逼迫: 新 run を止めて archive する。logger だけ無効にして走らない。
- 時刻逆転: NTP 再同期後、新しい run として再開する。古い CSV へ追記しない。

各日終了後、clean stop、logs、manifest、resolved profile、event、manual override を archive し hash を残す。翌日の parameter 更新前にその日の replay を行う。

14 出力の所在と追跡

path	内容
outputs/logs	measurement/live 同期 CSV
outputs/runtime	live raw/corrected weather、runtime state
outputs/identification	fit summary、residual、map、報告書

path	内容
simulation.output_dir	summary/detail/plan CSV、HTML、図、manifest
.run/solar_<mode>.log	launch stdout/stderr
rqt_graph_solar_<mode>.png	実 node graph

結果を人へ渡すときは、CSV 単体ではなく、manifest、resolved profile、map/evidence hash、solver status、警告を一組にする。latest_simulation_run.json は最新 run への安定 pointer で、generic filename の中身を推測してはならない。

15 教材と最終確認

- docs/complete_flow_workbook: live 1 周期から上位 MPC の有限差分、L-BFGS、有限 grid 証明まで。
- docs/mle_hand_calculation_workbook: Gaussian/Huber、PV、motion、battery、joint MLE、採否まで。
- docs/solar_all_in_one_manual: 物理モデル、全体構成、同定・運用の統合解説。
- docs/package_inventory: source/config/ROS topic の機械生成 inventory。

unit test や optimizer 成功だけを、実車安全または高精度の保証にはしてはならない。採用条件は、独立 data、制約付き全コース sim、実 ROS 起動、通信異常試験、shadow road test を すべて通すことである。